React Routing Demo 🚀

Una aplicación de demostración de routing en React que implementa un sistema de gestión de productos y proveedores usando React Router v6 y Context API.

📋 Tabla de Contenidos

- Requisitos Previos
- Instalación
- Estructura del Proyecto
- Guía de Implementación
- Características
- Rutas Disponibles

Requisitos Previos

- Node.js y npm instalados
- Conocimientos básicos de React y TypeScript
- Familiaridad con React Router

Instalación

```
npm install
npm  i react-router-dom
npm i bootstrap@latest
npm run dev
```

📁 Estructura del Proyecto

```
src/
├── components/          # Componentes reutilizables
│   └── Navbar.tsx      # Barra de navegación principal
├── context/            # Context API para gestión de estado
│   ├── ProductContext.tsx
│   └── SupplierContext.tsx
├── data/               # Datos iniciales
│   ├── products.ts
│   └── suppliers.ts
├── pages/              # Componentes de página
│   ├── Home.tsx
│   └── products/
│       ├── ProductList.tsx
│       └── ProductsForm.tsx
```

```
├── types/              # Definiciones de TypeScript
│   ├── Product.ts
│   └── Supplier.ts
├── App.tsx             # Componente principal
├── main.tsx            # Punto de entrada
└── index.css           # Estilos globales
```

🔨 Guía de Implementación

1. Definir los Tipos

Primero, creamos los tipos de datos que utilizará nuestra aplicación.

Archivo: src/types/Product.ts

```
export type Product = {
    id? : number;
    name: string;
    category : string;
    price : number;
}
```

Define la estructura de un producto con propiedades opcionales para el ID.

---

2. Crear Datos Iniciales

Preparamos los datos de prueba para productos y proveedores.

Archivo: src/data/products.ts

```
import { type Product } from "../types/Product";

export const products: Product[] = [
    { id: 1, name: "Kayak", category: "Watersports", price: 1200.98 },
    { id: 2, name: "Soccer Ball", category: "Soccer", price: 80 },
    { id: 3, name: "Trail Shoes", category: "Running", price: 150 },
    { id: 4, name: "Thermal Hat", category: "Running", price: 15.90 },
    { id: 5, name: "Heated Gloves", category: "Running", price: 21.30 }
];
```

Este archivo exporta un array inicial de productos que será usado como dato por defecto en el contexto.

---

3. Crear la Página Home

Archivo: src/pages/Home.tsx

```
const Home = () => {
    return (
    <div className="text-center">
        <h2>Wellcome</h2>
        <p>React Routing Demo</p>
    </div>
    )
}

export default Home;
```

La página principal de la aplicación que se muestra al acceder a la ruta raíz (`/`).

---

4. Crear la Barra de Navegación

La barra de navegación utiliza el componente `NavLink` de React Router, que es ideal para navegación porque añade automáticamente la clase `active` cuando la ruta coincide.

Archivo: src/components/Navbar.tsx

```
import { NavLink } from "react-router-dom"

const NavBar = () => {
    return (
        <nav className="navbar navbar-expand-lg navbar-dark bg-dark text-white">
            <div className="container">
                <span className="navbar-brand">Routing Demo</span>
                <NavLink className="nav-link" to="/">Home</NavLink>
                <NavLink className="nav-link" to="/products">Products</NavLink>
                <NavLink className="nav-link" to="/suppliers">Suppliers</NavLink>
            </div>
        </nav>
    )
}

export default NavBar;
```

Características:

- `NavLink` proporciona navegación activa automáticamente
- `to` especifica la ruta destino
- El componente se importa en src/App.tsx para ser mostrado en todas las páginas

---

5. Crear el Context de Productos

El Context API permite compartir estado entre componentes sin necesidad de pasar props a través de múltiples niveles.

Archivo: src/context/ProductContext.tsx

```
import React, { useState } from "react";
import type { Product } from "../types/Product";
import {products as initialProducts} from "../data/products";

type ProductContextType = {
    products : Product[];
    addProduct: (p: Product) => void;
    updateProduct: (p: Product) => void;
    deleteProduct: (id: number) => void;
}

export const ProductContext = React.createContext<ProductContextType>(
    {} as ProductContextType
)

export const ProductProvider = ({children} : {children: React.ReactNode}) => {
    const [products, setProducts] = useState<Product[]>(initialProducts);

    const addProduct = (product: Product) => {
        setProducts(prev => [...prev, {...product, id: Date.now()}]);
    }

    const updateProduct = (product: Product) => {
        setProducts(prev => prev.map(p => p.id === product.id ? product : p));
    }

    const deleteProduct = (id: number) => {
        setProducts(prev => prev.filter(p => p.id !== id));
    }

    return (
        <ProductContext.Provider value={{products, addProduct, updateProduct,
deleteProduct}}>
            {children}
        </ProductContext.Provider>
    );
}
```

Explicación:

- `ProductContextType`: Define la forma del contexto con la lista de productos y tres operaciones CRUD:
  - `addProduct`: Añade un nuevo producto con un ID basado en la fecha actual
  - `updateProduct`: Busca un producto por ID y actualiza sus datos
  - `deleteProduct`: Filtra el array eliminando el producto con el ID especificado
- `ProductContext`: Se crea con `React.createContext()` tipado como `ProductContextType`
- `ProductProvider`:

  - Es un componente envolvente que gestiona el estado de productos
  - Recibe `children` (componentes hijos) para que puedan acceder al contexto
  - Proporciona el valor con todos los datos y métodos mediante `ProductContext.Provider`

Operaciones:

- **Crear**: Añade a la lista anterior y genera un ID único con `Date.now()`
- **Actualizar**: Mapea la lista buscando por ID y reemplaza el producto encontrado
- **Eliminar**: Filtra la lista manteniendo todos excepto el que tiene el ID especificado

---

6. Crear el Context de Proveedores

Implementamos exactamente lo mismo para los proveedores.

Archivo: src/context/SupplierContext.tsx

> Sigue la misma estructura que src/context/ProductContext.tsx

---

7. Crear el Formulario de Productos

Este componente maneja tanto la creación como la edición de productos.

Archivo: src/pages/products/ProductsForm.tsx

```
import { useContext, useState } from "react";
import { useNavigate, useParams } from "react-router-dom";
import { ProductContext } from "../../context/ProductContext";

const ProductsForm = () => {
    const {products, addProduct, updateProduct} = useContext(ProductContext);
    const {id}= useParams()
    const navigate = useNavigate();

    const productToEdit = products.find(p=> p.id === Number(id));

    const [name,setName] = useState(productToEdit ? productToEdit.name : '');
    const [price,setPrice] = useState(productToEdit ? productToEdit.price : 0);
    const [category, setCategory] = useState(productToEdit ?
productToEdit.category : '');

    const handleSubmit = (e: React.FormEvent) => {
        e.preventDefault();

        id ? updateProduct({id: Number(id), name, price, category})
        : addProduct({name, price, category});

        navigate("/products")
    }

    return(
```

```
        <form onSubmit={handleSubmit}>
            <h2>{id ? 'Edit Product' : 'New Product'}</h2>
            <input type="text"
             className="form-control mb-2"
              placeholder="Name"
               value={name}
                onChange={e=> setName(e.target.value)}
                 required />

            <input type="number"
             className="form-control mb-2"
              placeholder="Price"
               value={price}
                onChange={e=> setPrice(Number(e.target.value))}
                 required />

            <input type="text"
             className="form-control mb-2"
              placeholder="Category"
               value={category}
                onChange={e=> setCategory(e.target.value)}
                 required />
            <button type="submit" className="btn btn-success">Save</button>
            <button type="button" className="btn btn-secondary"
             onClick={() => navigate("/products")}>Cancel</button>
        </form>
    )
}

export default ProductsForm;
```

Explicación detallada:

1. **Recuperar datos del contexto**: `useContext(ProductContext)` proporciona acceso a productos y métodos CRUD
2. **Obtener parámetros de URL**: `useParams()` extrae el `id` de la URL (ej: `/products/123/edit`)
3. **Navegación programática**: `useNavigate()` permite cambiar de página mediante código
4. **Búsqueda del producto a editar**: Usa `find()` para localizar el producto por ID
5. **Estados de formulario**: Inicializa los estados con datos del producto si existe, o vacíos si es nuevo
6. **Manejo del envío**:
    - Si hay `id`: Actualiza el producto existente
    - Si no hay `id`: Crea un nuevo producto
    - Después de guardar, navega a la lista de productos (`/products`)

---

8. Crear la Lista de Productos

Archivo: src/pages/products/ProductList.tsx

```
import { use } from "react"
import { ProductContext } from "../../context/ProductContext";
import { Link, useNavigate } from "react-router-dom";

const ProductList = () => {
    const {products,deleteProduct} = use(ProductContext);
    const navigate = useNavigate()

    return (
        <div>
            <h2>Product List</h2>
            <button className="btn btn-primary mb-2"
            onClick={() => navigate("/products/new")}>Add Product</button>
            <ul className="list-group">
                {products.map(p =>(
                    <li key={p.id} className="list-group-item d-flex justify-
content-between align-items-center">
                        {p.name}
                        <div>
                            <Link to={`/products/${p.id}/edit`}
                            className="btn btn-sm btn-warning me-2">
                            Edit
                            </Link>
                        </div>
                        <button className="btn btn-sm btn-danger"
                        onClick = {()=>{
                            const ok = window.confirm("Are you sure you want to
delete this product?");
                            if(ok) deleteProduct(p.id!);
                        }
                        }
                        >
                            Delete
                        </button>
                        </li>
                ))}
            </ul>
        </div>
    )
}

export default ProductList;
```

Características:

- **Botón "Add Product"**: Navega a `/products/new` para crear un nuevo producto
- **Recorrido de lista**: `map()` itera sobre todos los productos
- **Botón "Edit"**: Utiliza `Link` para navegar a `/products/{id}/edit` pasando el ID como parámetro
- **Botón "Delete"**: Pide confirmación con `window.confirm()` antes de eliminar
- **Key en elementos**: Cada `li` tiene `key={p.id}` para que React identifique los elementos correctamente

---

9. Configurar el Routing

El routing se configura en dos lugares:

A. En src/main.tsx - Configuración Global

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.tsx'
import { BrowserRouter } from 'react-router-dom'
import {ProductProvider } from './context/ProductContext.tsx'
import { SupplierProvider } from './context/SupplierContext.tsx'
import 'bootstrap/dist/css/bootstrap.min.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <BrowserRouter>
      <ProductProvider>
        <SupplierProvider>
        <App />
        </SupplierProvider>
      </ProductProvider>
    </BrowserRouter>
  </StrictMode>,
)
```

Orden de envolvimiento (de adentro hacia afuera):

1. `BrowserRouter`: Habilita el enrutamiento en toda la aplicación
2. `ProductProvider`: Proporciona acceso al contexto de productos
3. `SupplierProvider`: Proporciona acceso al contexto de proveedores
4. `App`: El componente principal de la aplicación

B. En src/App.tsx - Definición de Rutas

```
import { Route, Routes } from "react-router-dom"
import NavBar from "./components/Navbar"
import Home from "./pages/Home"
import ProductList from "./pages/products/ProductList"
import ProductsForm from "./pages/products/ProductsForm"

function App() {
  return (
    <>
      <NavBar/>
      <div className="container mt-4">
        <Routes>
          <Route path="/" element={<Home/>}/>
          <Route path="/products" element={<ProductList/>}/>
```

```
            <Route path="/products/new" element={<ProductsForm/>}/>
            <Route path="/products/:id/edit" element={<ProductsForm/>}/>
          </Routes>
        </div>
      </>
    )
  }

  export default App
```

Rutas definidas:

- `/`: Página de inicio
- `/products`: Lista de productos
- `/products/new`: Formulario para crear producto
- `/products/:id/edit`: Formulario para editar producto (el `:id` es un parámetro dinámico)

Nota importante: Aún falta añadir las rutas para `Suppliers` siguiendo el mismo patrón.

✨ Características

- ☑ Routing con React Router v6
- ☑ Gestión de estado con Context API
- ☑ Operaciones CRUD completas (Create, Read, Update, Delete)
- ☑ Formularios controlados con React Hooks
- ☑ Tipado completo con TypeScript
- ☑ Navegación dinámica con `useParams()` y `useNavigate()`
- ☑ Interfaz con Bootstrap 5
- ☑ Confirmaciones de acción del usuario

🗺 Rutas Disponibles

Ruta	Componente	Descripción
`/`	`Home`	Página de inicio
`/products`	`ProductList`	Lista de todos los productos
`/products/new`	`ProductsForm`	Formulario para crear nuevo producto
`/products/:id/edit`	`ProductsForm`	Formulario para editar producto existente
`/suppliers`	`SupplierList`	Lista de proveedores (pendiente de implementar)

📌 Conceptos Clave

React Router

- **`NavLink`**: Similar a `Link`, pero añade clases CSS cuando la ruta es activa

- `Link`: Navega a una ruta sin recargar la página
- `useNavigate()`: Hook para navegar programáticamente
- `useParams()`: Hook para acceder a parámetros dinámicos de la URL
- `Routes` **y** `Route`: Definen las rutas y los componentes asociados

Context API

- `createContext()`: Crea un contexto para compartir datos
- `useContext()`: Hook para acceder a un contexto
- **Provider**: Componente que proporciona los datos del contexto a sus hijos

TypeScript

- **Tipos personalizados**: Define la estructura de datos con `type`
- **Tipos opcionales**: Usa `?` para propiedades que pueden ser undefined
- **Tipado de React**: `React.FormEvent`, `React.ReactNode`, etc.

🚀 Próximos Pasos

1. Implementar las rutas completas para `Suppliers`
2. Añadir validación avanzada de formularios
3. Persistencia de datos con localStorage o una API
4. Mejorar estilos y responsive design
5. Añadir filtrado y búsqueda en las listas

Última actualización: 19 de enero de 2026